

Chatbot Orchestration Workflow

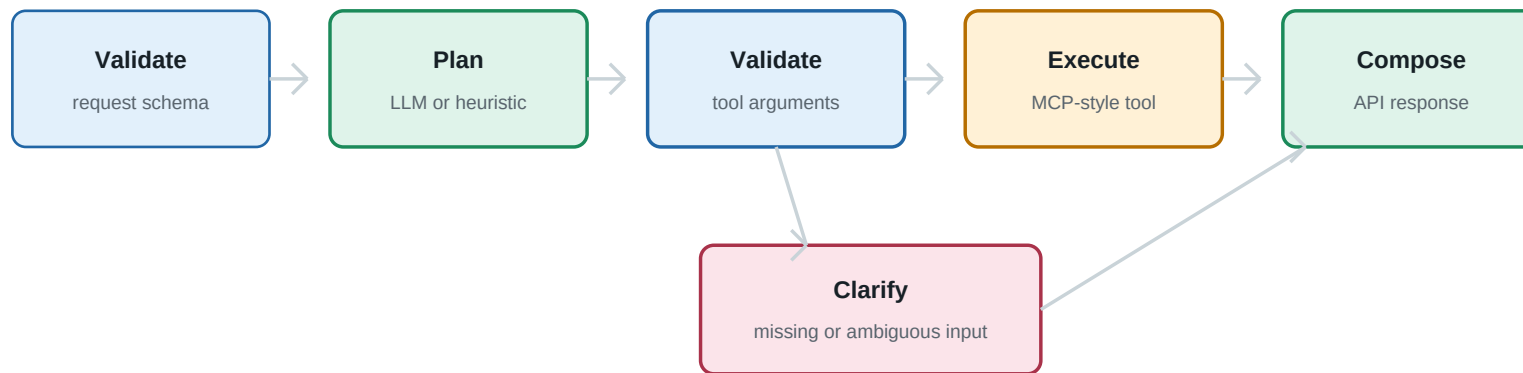
DL-06-T12

Prototype and implementation overview for DiscountMate's LangGraph-based DL-06 chatbot.

LangGraph + LangFlow prototype

This implementation converts the combined DiscountMate chatbot from a custom planner and runner into a graph-based workflow. The public API remains stable while the internal execution path becomes easier to inspect, test, and extend.

Workflow Visual

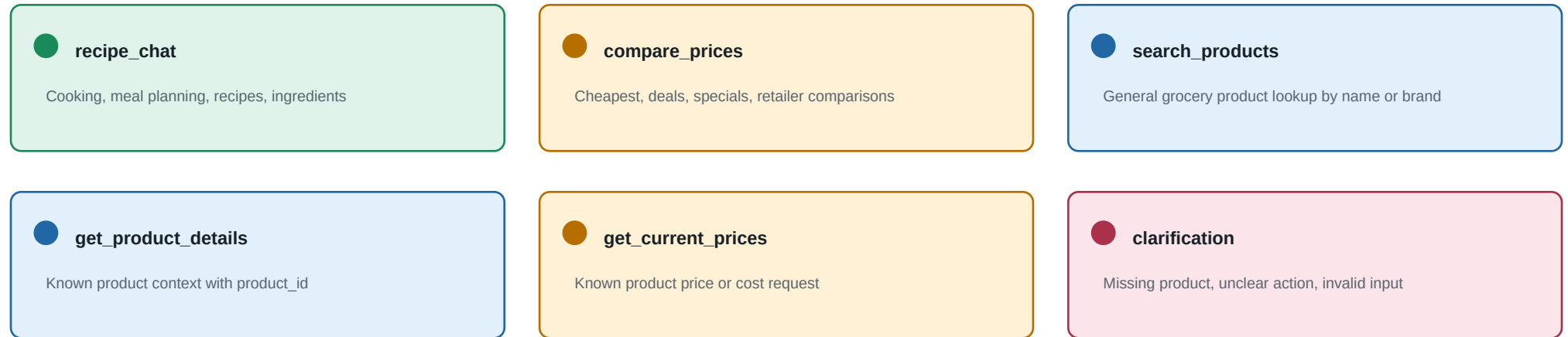


The Flask endpoint keeps the existing AgentResponse contract while graph nodes handle routing and tool execution.

The graph preserves deterministic fallback behavior for local testing and uses LangGraph when the dependency is installed in the ML service environment.

Routing And Tool Coverage

Tool routing coverage



Scenario	Selected route	Result
Recipe or meal planning request	recipe_chat	Calls the existing Recipe RAG pipeline and returns recipe answer data.
Cheapest, deals, specials, or comparison request	compare_prices	Searches product candidates, resolves prices, and returns the cheapest retailer.
Known product context with detail intent	get_product_details	Returns display-ready product metadata.
Known product context with price intent	get_current_prices	Returns current valid prices filtered by retailer when provided.
Unclear or incomplete request	clarification	Returns a structured follow-up question without executing a risky tool call.

Implementation Summary

API layer

POST /api/ml/chatbot/chat keeps proxying to the Flask ML service.

Agent wrapper

DiscountMateAgent remains the public entry point used by app.py.

Graph workflow

LangGraph StateGraph owns validation, routing, execution, and response composition.

Tool layer

Existing MCP-style product, price, comparison, and recipe tools are reused unchanged.

Area	What changed
Runtime workflow	Added chatbots/agents/langgraph_workflow.py with StateGraph nodes for validation, planning, tool execution, clarification, and answer composition.
Compatibility	Kept DiscountMateAgent.chat(payload) as the stable entry point used by the Flask route.
Dependencies	Added LangGraph to the ML service runtime requirements. LangFlow remains optional for local visual prototyping.
Visual prototyping	Added chatbots/langflow/discountmate_chatbot_flow.json and a local LangFlow README.
Verification	Ran Python syntax checks, chatbot smoke tests, and JSON validation for the flow artifact.

Next practical step: install the updated ML service dependencies, then run the same smoke test again to confirm the workflow backend reports LangGraph instead of local-fallback.